

INTRANET 1 - LDAPS

Auteur

Nom : [DEMAISON Camille]

Dans le cadre du projet nous devons déployer sur un **docker** de GNS3 un serveur avec **Flask ou PHP** pour le backend et **njinx ou apache** pour gérer les requêtes.

Dans le cahier des charges il est demandé que :

1. Le site web intranet soit développé avec la technologie de votre choix et doit proposer un système de publication des actualités de l'entreprise avec authentification LDAP.
2. Le tout soit développé en respectant les bonnes pratiques en termes de sécurité. Les actualités doivent être visibles sur la page d'accueil du site et seront classées par date de publication.
3. Soit mis en place un certificat SSL signé par l'autorité de certification interne pour sécuriser les échanges avec le serveur.
4. Le tout doit être déployé par ansible et doit fonctionner automatiquement.

D'abord on a développé une application web sécurisée construite avec **Flask** et on a géré l'authentification. L'application utilise une **authentification LDAP (Active Directory)** via **LDAPS** (SSL), et un design frontend avec **Bootstrap**.

Le contenu de l'application va évoluer au cours du développement des différentes composantes. La documentation sera divisé en 4 parties pour chaque composantes.

Structure du projet :

Actuellement l'application est situé dans un dossier nommé `/opt/flask_app/` avec cette structure :

```
/opt/flask_app/
├─ app.py                # Application principale Flask
├─ requirements.txt      # Listes de bibliothèques à installer
├─ templates/
│   ├─ login.html        # Page de connexion (Bootstrap)
│   └─ home.html         # Page d'accueil après authentification
└─ static/
    └─ favicon.ico       # Favicon des pages
```

Technologies utilisées :

Technologie	Usage
Flask	Framework web principal
Flask-Login	Gestion des sessions utilisateurs
Flask-LDAP3-Login	Authentification via LDAP/LDAPS
ldap3	Connexion sécurisée via SSL/TLS
Bootstrap 5	Interface utilisateur moderne et réactive

Bibliothèques installées

On a actuellement besoin d'installer ces bibliothèques pour faire fonctionner cette partie du projet :

```
Flask
flask-login
flask-ldap3-login
ldap3
```

Certificat pour LDAPS

Comme vu dans les documentations sur déploiement des CA, tout est géré avec ansible.

On doit tout de même faire attention à certains paramètres :

- Le certificat CA doit être fourni dans le chemin `/usr/local/share/ca-certificates/`
- Le port 636 doit être ouvert entre le conteneur et le serveur AD
- Le nom `srv-ad01.itway.local` doit correspondre au **CN/SAN** du certificat du serveur AD

app.py — Application principale

L'application principale utilise un framework Python nommé [Flask](#) pour gérer le backend, il permet d'avoir des **outils automatisés** au lieu de devoir tout gérer soit même en PHP.

On utilise le LDAP pour gérer la connexion des utilisateurs. Cela permet d'utiliser les **identifiants de connexion des sessions windows** pour accéder au site web.

Importation des bibliothèques

Dans un premier temps on importe tout les bibliothèques nécessaires pour faire fonctionner cette partie du projet.

```
from flask import Flask, render_template, redirect, flash, url_for, request
from flask_ldap3_login import LDAP3LoginManager
from flask_login import LoginManager, login_user, UserMixin, current_user, logout_user, login_required
from flask_ldap3_login.forms import LDAPLoginForm
from ldap3 import Tls
import ssl
```

Initialisation de l'application Flask

Les éléments `app.config['OPTION']` permettent de configurer des paramètres de Flask.

```
app = Flask(__name__)
app.config['SECRET_KEY'] = 'secret'
app.config['DEBUG'] = True
```

- **SECRET_KEY = 'secret'** => Clé secrète pour la gestion des sessions

Elle est générée une fois avec cette commande dans un shell :

```
python3 -c "import secrets; print(secrets.token_hex(32))"
```

Puis il suffit de la mettre dans le code à la place de `'secret'`.

Générer une bonne clé permet d'éviter une attaque par **cookie tampering** (attaque par modification de cookies) en chiffrant les cookies de session. Cela permet aussi de protéger les formulaires contre les attaques CSRF par un champ caché avec un token que Flask vérifie quand on envoie le formulaire.

- **DEBUG = True** => Permet d'activer le mode debug

Warning

`app.config['DEBUG'] = True` => A ne pas conserver en production car cela affiche les messages d'erreurs sur le navigateur.

Tip

Pour tout ce qui est données sensibles comme la `SECRET_KEY` il est conseillé de les mettre dans des variables d'environnement.

Ce n'est pas mis en place ici par soucis de simplicité, mais cela permet de pouvoir stocker ses fichiers (comme dans un git) sans afficher les données sensibles en clair.

Configuration LDAP pour une connexion securisee via LDAPS (port 636)

En Flask, on peut définir tous les paramètres de configuration de l'application (comme les mots de passe, les infos LDAP, etc.).

```
app.config['LDAP_HOST'] = 'srv-ad01'
app.config['LDAP_PORT'] = 636
app.config['LDAP_BASE_DN'] = 'dc=itway,dc=local'
app.config['LDAP_USER_LOGIN_ATTR'] = 'sAMAccountName'
app.config['LDAP_USER_SEARCH_SCOPE'] = 'SUBTREE'
app.config['LDAP_BIND_USER_DN'] = 'cn=Liaison
LDAP,ou=Users,ou=IT,ou=Service iTWay,dc=itway,dc=local'

app.config['LDAP_BIND_USER_PASSWORD'] = 'secret'
app.config['LDAP_USE_SSL'] = True
app.config['LDAP_ADD_SERVER'] = False
```

- **LDAP_HOST** = 'srv-ad01' => Adresse du serveur AD qui contient le LDAP
- **LDAP_PORT** = 636 => Port coté serveur AD pour la connection LDAPS
- **LDAP_BASE_DN** = 'dc=itway,dc=local' => DN de base pour la recherche des utilisateurs (*du style itway.local*)
- **LDAP_USER_LOGIN_ATTR** = 'sAMAccountName' => Ici on spécifie qu'on utilise l'identifiant de connexion windows pour se connecter sur le site (sinon ça aurait été le nom de la session)
- **LDAP_USER_SEARCH_SCOPE** = 'SUBTREE' => Portée de la recherche dans l'annuaire, on permet de chercher dans les sous dossier (à partir de la racine par défaut)
- **LDAP_BIND_USER_DN** = 'cn=Liaison LDAP,ou=Users,ou=IT,ou=Service iTWay,dc=itway,dc=local'

=> C'est le **compte technique** que Flask utilise pour **se connecter au LDAP** et **rechercher les utilisateurs**.

- Ce compte doit exister dans notre AD.
- Il a le **droit de lire l'annuaire** mais pas besoin d'être un admin. Un compte lecteur simple suffit.

- **LDAP_BIND_USER_PASSWORD** = 'secret' => Mot de passe du compte lecteur de la liaison LDAP
- **LDAP_USE_SSL** = True => Active et oblige la **communication par connexion SSL**
- **LDAP_ADD_SERVER** = False => On désactive l'ajout dynamique du serveur LDAP pour ajouter manuellement le serveur LDAP avec TLS (vu plus tard)

Tip

Pour debug : `apt update && apt install ldap-utils -y`

Puis :

```
ldapsearch -x -H ldap://srv-ad01 -D "cn=Liaison
LDAP,ou=Users,ou=IT,ou=Service iTway,dc=itway,dc=local" -w
'Projet!WLC' -b "dc=itway,dc=local"
```

On peut adapter les paramètres pour faire des tests.

Initialisation des gestionnaires Flask-Login et Flask-LDAP3-Login

On initialise le **gestionnaire de login** avec l'application Flask, définit un message de redirection personnalisé et on définit ensuite la redirection automatique si non connecté (ici sur la page login).

On crée ensuite l'objet **gestionnaire de LDAP**, et enfin on initialise le gestionnaire de LDAP avec l'application Flask.

```
login_manager = LoginManager(app)
login_manager.login_message = "Vous devez être connecté pour accéder à
cette page."
login_manager.login_view = 'login'
ldap_manager = LDAP3LoginManager()
ldap_manager.init_app(app)
```

Configuration TLS

D'abord on configure le TLS (**Transport Layer Security**) pour sécuriser la connexion LDAP.

```
tls_ctx = Tls(
    validate=ssl.CERT_REQUIRED,
    version=ssl.PROTOCOL_TLSv1_2,
    ca_certs_file='/usr/local/share/ca-certificates/myCA.crt',
    valid_names=['srv-ad01.itway.local']
)
```

Dans cette configuration on a :

- La **vérification du certificat serveur** qui est obligatoire
- La **version TLS** qui sera utilisée
- Le **chemin vers le certificat** de l'autorité de certification
- Le **nom DNS** attendu dans le certificat serveur

Comme on a désactivé l'ajout automatique du serveur pour paramétrer le TLS, on doit ajouter le serveur LDAP avec notre configuration TLS.

```
with app.app_context():
    ldap_manager.add_server(
        app.config['LDAP_HOST'],
        app.config['LDAP_PORT'],
        app.config['LDAP_USE_SSL'],
        tls_ctx=tls_ctx
    )
```

Gestion des utilisateurs

Ici on a nos différents outils de gestion des utilisateurs. On a un dictionnaire `users` en mémoire pour stocker les utilisateurs connectés. Ils seront enregistrés dans la RAM lorsque le programme sera lancé et donc libérés si l'on stoppe le programme.

```

users = {}

class User(UserMixin):
    def __init__(self, dn, username, data):
        self.dn = dn # DN (distinguished name) de l'utilisateur
        self.username = username # Identifiant utilisateur
        self.data = data # Donnees LDAP brutes associees a l'utilisateur

    def get_id(self):
        return self.dn # Utilise le DN comme identifiant unique

```

On utilise la classe utilisateur compatible avec Flask-Login fournis par la documentation officielle. On a donc à déclarer le constructeur `__init__` pour **créer l'objet** et la méthode `get_id` pour **avoir l'ID de l'objet**.

```

@login_manager.user_loader
def load_user(id):
    return users.get(id)

```

On a ici la fonction pour recharger l'utilisateur à partir de son ID.

```

@ldap_manager.save_user
def save_user(dn, username, data, memberships):
    required_group = 'CN=GRP-INTRANET,OU=Groupes,OU=Service
ITway,DC=itway,DC=local'

    if 'memberOf' in data and required_group in data['memberOf']:
        user = User(dn, username, data)
        users[dn] = user # Sauvegarde en memoire
        return user

    # Refus si l'utilisateur ne fait pas partie du groupe autorise
    return None

```

Ici on a la fonction appelée lors d'une authentification LDAP réussie.

- On déclare le groupe requis pour avoir accès à l'application puis on vérifie l'appartenance au groupe.
 - Si c'est le cas alors on l'ajoute au dictionnaire puis on retourne l'utilisateur, sinon on ne l'enregistre pas et on retourne `None`.
-

Message d'erreur

Un message d'erreur est envoyé et la page est rechargé si le serveur rencontre une erreur.

```
# Message d'erreur lors de la levee d'une erreur
@app.errorhandler(Exception)
def handle_error(e):
    flash("Une erreur s'est produite. Veuillez réessayer ou contacter
l'administrateur.", "error")
    return redirect(request.url)
```

Route : Page d'accueil

Ensuite on a les différents chemins sur les URLs de notre site web. Le premier défini ici est le chemin par défaut.

```
@app.route('/')
@login_required
def home():
    return render_template('home.html', user=current_user)
```

Son fonctionnement est assez simple, si l'utilisateur n'est pas connecté avec ses identifiants alors il est redirigé sur la page de login sinon il a donc accès à la page principale qui est ici `home.html`.

Route : Page de connexion

Le second chemin défini ici est celui pour se connecter.

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    if current_user.is_authenticated:
        return redirect(url_for('home'))

    form = LDAPLoginForm()
```

```

    if form.is_submitted():
        if form.validate():
            user = form.user
            if user:
                login_user(user)
                next_page = request.args.get('next') # Récupère l'URL de
redirection
                return redirect(next_page or url_for('home'))
            else:
                flash("Vous n'avez pas les droits pour accéder à cette
application.")
        else:
            flash("Nom d'utilisateur ou mot de passe incorrect.")

    return render_template('login.html', form=form)

```

On définit les méthodes de la route :

- `methods=[ 'GET', 'POST' ]` : Cette route accepte deux méthodes HTTP :
 - GET => Pour **recupérer et afficher le formulaire vide**
 - POST => Pour **envoyer les données du formulaire** (lorsqu'on clique sur "Connexion")

Puis dans un premier temps on vérifie si l'utilisateur **est déjà authentifié**, si c'est le cas alors on le redirige vers l'accueil sinon on continue.

D'abord on initialise un formulaire avec : `form = LDAPLoginForm()`

Puis on vérifie si un utilisateur **a envoyé un formulaire** avec la méthode `'POST'` sinon on lui affiche la page de connexion.

Ensuite on vérifie si l'utilisateur **a mis les bons identifiants** (si le lecteur LDAP a trouvé l'utilisateur) sinon on lui affiche la page de connexion avec un message le prévenant.

On continue en vérifiant s'il **appartient au bon groupe** (permet de filtrer par un groupe de personnes autorisées sur l'AD) sinon on lui affiche la page de connexion avec un message le prévenant.

Si tout est bon alors il sera enregistré dans le dictionnaire des personnes connectées et redirigé vers la page où il était ou la principale.

Info

Si un utilisateur a été redirigé vers cette page de connection car il n'était pas identifié alors le lien ressemblera à ceci :

```
/login?next=%2F
```

C'est tout à fait normal, il est indiqué que cette page doit être rechargé une fois identifié (%2F signifiant le symbole /)

Route : La déconnexion

Le dernier chemin défini ici est celui pour se déconnecter.

```
@app.route('/logout')
def logout():
    logout_user()
    flash("Vous vous êtes déconnecté avec succès.")
    return redirect(url_for('login'))
```

Son principe est lui aussi très simple, lorsque l'utilisateur est sur ce chemin alors il est déconnecté puis redirigé vers la page de connection.

Lancement de l'application Flask

```
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Actuellement on lance l'application Flask sur 0.0.0.0:5000 , donc **accessible par toutes les machines du réseau** sur le port 5000. Elle écoute actuellement sur toutes les interfaces.

⚠ Warning

Les **principales problématiques** sur ce fichier ont été de trouver les **bons paramètres** à utiliser pour Flask et de les configurer correctement.

En utilisant des **outils de debug** disponibles sur la [documentation officielle](#), nous avons pu sectoriser les problèmes rencontrés et les corriger.

Exemple : `app.config['DEBUG'] = True` ou bien même en affichant des variables sur le serveur web avec un `print()`

login.html — Page web de connexion

Nos pages web utilisent les feuilles de style de bootstrap pour avoir un aspect responsive et plus agréable pour l'utilisateur. Cela rends le développement plus rapide même si l'on a redéfini quelques parties CSS dans la partie `<head>...</head>`. Une favicon (image pour les raccourcis et onglets) a aussi été ajouté.

```
<!doctype html>
<html lang="fr">
  <head>
    <meta charset="utf-8">
    <title>Connexion</title>
    <link rel="icon" href="{{ url_for('static', filename='favicon.ico')
  }}" type="image/x-icon">
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.
css" rel="stylesheet">
    <style>
      body {
        background: #f0f2f5;
        display: flex;
        justify-content: center;
        align-items: center;
        height: 100vh;
      }
      .login-form {
        background: white;
        padding: 2rem;
        border-radius: 10px;
        box-shadow: 0 0 10px rgba(0,0,0,0.1);
```

```

        width: 100%;
        max-width: 400px;
    }
</style>
</head>

<body>
    <div class="login-form">
        <h3 class="text-center mb-4">Connexion Intranet</h3>
        {% with messages = get_flashed_messages() %}
            {% if messages %}
                <div class="alert alert-warning">{{ messages[0] }}</div>
            {% endif %}
        {% endwith %}
        <form method="POST">
            {{ form.hidden_tag() }}
            <div class="mb-3">
                {{ form.username.label(class="form-label") }}
                {{ form.username(class="form-control") }}
            </div>
            <div class="mb-3">
                {{ form.password.label(class="form-label") }}
                {{ form.password(class="form-control") }}
            </div>
            <div class="d-grid">
                {{ form.submit(class="btn btn-primary") }}
            </div>
        </form>
    </div>
</body>
</html>

```

Dans la partie `<body>...</body>` on retrouve des divisions en cascades pour pouvoir organiser nos différents éléments.

Les parties avec des accolades sont des **instructions Jinja2**, le moteur de template utilisé par **Flask**. Ces parties sont sur les fichiers du serveur mais lors du chargement de la page ces parties sont **remplacées par du code HTML**.

Cela permet de :

- Cacher le fonctionnement logique de la page (en backend)
- Pouvoir utiliser des composantes de Flask (fonctions, variables, etc.)

Il y a deux types d'instructions utilisables :

- **{% LOGIQUE %}** => Ce sont des **instructions** (boucles, conditions, blocs...)
- **{{ VALEURS }}** => Ce sont des **expressions**, elles affichent des variables ou le résultat d'une fonction Python

On a donc différents éléments qui sont gérés :

- Affichage des messages d'erreurs si des erreurs sont remontées
- Formulaire "POST" qui contient :
 - Un **tag caché** qui est utilisé par Flask
 - Une zone de texte avec l'**username** (entrée obligatoire)
 - Une zone de texte avec le **mot de passe** (entrée obligatoire)

Info

La réponse à l'envoi du formulaire est lui géré par Flask donc rien de plus n'est nécessaire ici.

La méthode **GET** est utilisée lors du chargement de la page pour récupérer le formulaire vide avec le tag.

home.html — Page web principale

Actuellement la page principale n'affiche qu'un message de bienvenue ainsi que les informations sur l'utilisateur (via les instructions vu précédemment).

```
<!doctype html>
<html lang="fr">
  <head>
    <meta charset="utf-8">
    <title>Accueil</title>
    <link rel="icon" href="{{ url_for('static', filename='favicon.ico')
  }}" type="image/x-icon">

    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.
```

```

css" rel="stylesheet">

</head>
<body class="bg-light">
  <div class="container mt-5">
    <div class="card shadow-sm">
      <div class="card-body">
        <h2 class="card-title">Bienvenue {{ user.data.cn }}</h2>
        <p class="card-text">DN : {{ user.dn }}</p>
        <a href="{{ url_for('logout') }}" class="btn btn-danger">Se
déconnecter</a>

      </div>
    </div>
  </div>
</body>
</html>

```

Info

Cette page sera celle qui servira pour l'affichage des posts, elle sera donc amené à évoluer.

Tester l'application Flask

Pour tester l'application il ne reste plus qu'à installer les différentes bibliothèques. J'ai mis la liste des bibliothèques dans le fichier `requirements.txt`.

```

apt update && apt install python3-pip -y

pip install --ignore-installed --no-cache-dir -r
/opt/flask_app/requirements.txt --break-system-packages

```

Puis pour lancer l'application :

```
python3 /opt/flask_app/app.py
```

Il ne reste plus qu'à tester sur un navigateur web d'une autre machine du réseau via : `http://srv-intranet:5000`.

Tests et vérifications

Une page temporaire a été créé pour les tests, une route basique a été ajouté dans **app.py**:

```
@app.route('/test')
@login_required
def test():
    return render_template('test.html', user=current_user)
```

Différents tests ont été procédé :

- Test sur la redirection des pages et messages d'erreurs :

On se connecte sur la page de test :

- <http://srv-intranet:5000/test>

Comme on n'est pas connecté alors on est redirigé vers :

- <http://srv-intranet:5000/login?next=%2Ftest>

Sur la page de login on peut **tester les différents messages d'erreurs**, on constate qu'ils fonctionnent bien.

- Un pour les mauvais ID donnés
- Un pour les ID n'appartenant pas au groupe

Une fois connecté on est bien redirigé vers la page initiale :

- <http://srv-intranet:5000/test>

- Vérification de la communication sur TLS avec le LDAP (LDAPS)

Pour cela on a utilisé Wireshark et l'on a fait une capture au moment de l'identification d'un client:

No.	Time	Source	Destination	Protocol	Length	Info
26	13.629853	192.168.20.4	192.168.20.2	TCP	74	57589 -> 636 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3389589724 TSecr=0 WS=128
27	13.630639	192.168.20.2	192.168.20.4	TCP	74	636 -> 57589 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM TSval=253875608 TSecr=3389589724
29	13.630743	192.168.20.4	192.168.20.2	TCP	66	57589 -> 636 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=3389589725 TSecr=253875608
30	13.630616	192.168.20.4	192.168.20.2	TLSv1.2	254	Client Hello
31	13.649513	192.168.20.2	192.168.20.4	TCP	66	636 -> 57589 [ACK] Seq=1 Ack=189 Win=2097920 Len=0 TSval=253875627 TSecr=3389589728
32	13.654770	192.168.20.2	192.168.20.4	TLSv1.2	1038	Server Hello, Certificate, Server Key Exchange, Certificate Request, Server Hello Done
33	13.654925	192.168.20.4	192.168.20.2	TCP	66	57589 -> 636 [ACK] Seq=189 Ack=973 Win=64128 Len=0 TSval=3389589749 TSecr=253875627
34	13.660727	192.168.20.4	192.168.20.2	TLSv1.2	236	Certificate, Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
35	13.662938	192.168.20.2	192.168.20.4	TLSv1.2	117	Change Cipher Spec, Encrypted Handshake Message
36	13.664118	192.168.20.4	192.168.20.2	TLSv1.2	184	Application Data
37	13.669209	192.168.20.2	192.168.20.4	TLSv1.2	117	Application Data
38	13.670345	192.168.20.4	192.168.20.2	TLSv1.2	156	Application Data
39	13.671274	192.168.20.2	192.168.20.4	TLSv1.2	238	Application Data
40	13.672296	192.168.20.4	192.168.20.2	TLSv1.2	373	Application Data
41	13.680554	192.168.20.2	192.168.20.4	TCP	1514	636 -> 57589 [ACK] Seq=1239 Ack=874 Win=2097152 Len=1448 TSval=253875664 TSecr=3389589766 [TCP segment of a reassembled...
42	13.686694	192.168.20.2	192.168.20.4	TCP	1514	636 -> 57589 [ACK] Seq=2687 Ack=874 Win=2097152 Len=1448 TSval=253875664 TSecr=3389589766 [TCP segment of a reassembled...
43	13.686618	192.168.20.2	192.168.20.4	TCP	1514	636 -> 57589 [ACK] Seq=4135 Ack=874 Win=2097152 Len=1448 TSval=253875664 TSecr=3389589766 [TCP segment of a reassembled...
44	13.686655	192.168.20.2	192.168.20.4	TCP	1514	636 -> 57589 [ACK] Seq=5583 Ack=874 Win=2097152 Len=1448 TSval=253875664 TSecr=3389589766 [TCP segment of a reassembled...
45	13.686672	192.168.20.2	192.168.20.4	TCP	1514	636 -> 57589 [ACK] Seq=7031 Ack=874 Win=2097152 Len=1448 TSval=253875664 TSecr=3389589766 [TCP segment of a reassembled...
46	13.686696	192.168.20.2	192.168.20.4	TCP	1514	636 -> 57589 [ACK] Seq=8479 Ack=874 Win=2097152 Len=1448 TSval=253875664 TSecr=3389589766 [TCP segment of a reassembled...
47	13.686707	192.168.20.2	192.168.20.4	TCP	1514	636 -> 57589 [ACK] Seq=9927 Ack=874 Win=2097152 Len=1448 TSval=253875664 TSecr=3389589766 [TCP segment of a reassembled...
48	13.686721	192.168.20.2	192.168.20.4	TCP	1514	636 -> 57589 [ACK] Seq=11375 Ack=874 Win=2097152 Len=1448 TSval=253875664 TSecr=3389589766 [TCP segment of a reassembled...
49	13.686741	192.168.20.2	192.168.20.4	TCP	1514	636 -> 57589 [ACK] Seq=12823 Ack=874 Win=2097152 Len=1448 TSval=253875664 TSecr=3389589766 [TCP segment of a reassembled...

On constate bien le handshake (échange de clés) puis la communication entre le serveur INTRANET et l'AD est **chiffré avec TLSv1.2**.

Pour l'instant seulement cette communication est chiffrée, pas celle entre le client web et le serveur INTRANET (car ils sont encore en HTTP).

Si on analyse le flux on obtiens bien un échange chiffré, même sur les paquets n'indiquant que TCP :

42	13.686604	192.168.20.2	192.168.20.4	TCP	1514	636	→	57589	[ACK]	Seq=2687	Ack=874	Win=2097152	Len=1448	TSval=		
43	13.686618	192.168.20.2	192.168.20.4	TCP	1514	636	→	57589	[ACK]	Seq=4135	Ack=874	Win=2097152	Len=1448	TSval=		
44	13.686655	192.168.20.2	192.168.20.4	TCP	1514	636	→	57589	[ACK]	Seq=5583	Ack=874	Win=2097152	Len=1448	TSval=		
45	13.686672	192.168.20.2	192.168.20.4	TCP	1514	636	→	57589	[ACK]	Seq=7031	Ack=874	Win=2097152	Len=1448	TSval=		
46	13.686696	192.168.20.2	192.168.20.4	TCP	1514	636	→	57589	[ACK]	Seq=8479	Ack=874	Win=2097152	Len=1448	TSval=		
47	13.686707	192.168.20.2	192.168.20.4	TCP	1514	636	→	57589	[ACK]	Seq=9927	Ack=874	Win=2097152	Len=1448	TSval=		
48	13.686721	192.168.20.2	192.168.20.4	TCP	Marquer/Démarquer le(s) paquet(s)									Win=2097152	Len=1448	TSval=
49	13.686741	192.168.20.2	192.168.20.4	TCP	Ignorer/Annuler le(s) paquet(s)									Win=2097152	Len=1448	TSval=
50	13.686755	192.168.20.2	192.168.20.4	TCP										Win=2097152	Len=1448	TSval=
51	13.686767	192.168.20.2	192.168.20.4	TCP	Fixer/Defixer le Temps de Référence									Seq=15719	Ack=874	Win=2097152
52	13.686812	192.168.20.4	192.168.20.2	TCP	Décalage dans le temps...									Win=62080	Len=0	TSval=33895
53	13.686830	192.168.20.4	192.168.20.2	TCP										Win=62080	Len=0	TSval=33895
54	13.686842	192.168.20.4	192.168.20.2	TCP	Commentaires sur les paquets									Win=64128	Len=0	TSval=3389
55	13.686852	192.168.20.4	192.168.20.2	TCP										Win=64128	Len=0	TSval=3389
56	13.687311	192.168.20.2	192.168.20.4	TCP	Editer Nom Résolu									Win=2097152	Len=1448	TSval=
57	13.687333	192.168.20.2	192.168.20.4	TLS	Appliquer comme un Filtre									Win=2097152	Len=1448	TSval=
58	13.687345	192.168.20.2	192.168.20.4	TCP	Préparer comme filtre									Win=2097152	Len=1448	TSval=
59	13.687368	192.168.20.2	192.168.20.4	TCP										Win=64128	Len=0	TSval=3389
60	13.687382	192.168.20.4	192.168.20.2	TCP	Filtre de Conversation									Win=2097152	Len=1448	TSval=
61	13.687396	192.168.20.2	192.168.20.4	TCP	Colorier la Conversation									Win=2097152	Len=1448	TSval=
				SCTP										Win=2097152	Len=1448	TSval=
														Win=2097152	Len=1448	TSval=

» [Conversation completeness: Complete, WITH_DATA (63)]

[TCP Segment Len: 1448]

Sequence Number: 9927 (relative sequence number)

Sequence Number (raw): 4292349333

[Next Sequence Number: 11375 (relative sequence number)]

Acknowledgment Number: 874 (relative ack number)

Acknowledgment Number (raw): 2141243354

1000 = Header Length: 32 bytes (8)

Flags: 0x010 (ACK)

Window: 8192

[calculated window size: 2097152]

Suivre

Copier

Préférences de protocole

Décoder comme...

Affiche Paquet dans Nouvelle Fenêtre

TCP Stream Ctrl+Alt+Maj+T

On obtient :

[illegible]

Il y a bien l'échange de clef puis les données sont chiffrées.

- Test d'injection LDAP :

On utilise des entrées du style :

Identifiant =>)(uid=))(|(uid=*

Mot de passe => 1=1;

On obtient comme résultat une erreur, ce qui affiche au client :

Une erreur s'est produite. Veuillez réessayer ou contacter l'administrateur.

Si l'on désactive cette fonctionnalité l'erreur apparait bien sur le serveur.

LDAPInvalidFilterError

Pour éviter cela on a ajouté une **fonction qui détecte les caractères spéciaux** :

```
def contains_ldap_special_chars(value):  
    return any(char in value for char in ['*', '(', ')', '\\'])
```

On a donc du ajouter cette vérification dans la route de login.

Si l'entrée contient un de ces caractères alors la fonction retourne `True` , dans ce cas on n'exécute pas la vérification LDAP et recharge la page avec un formulaire vierge.

La page est rechargé sans message pour que l'attaquant n'ai pas d'indice sur ce qu'il se passe coté serveur.

Maintenant l'erreur `LDAPInvalidFilterError` ne remonte plus car il n'y a **plus de requête LDAP malveillantes exécutée**.